PRACTICAL 4

Program1:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    int i;

    for(i = 0; i < 3; i++)
    {
        if(fork() == 0)
        {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            sleep(2);
            printf("Child %d Finished\n", i + 1);
            exit(0);
        }
    }

    for(i = 0; i < 3; i++)
    {
        wait(NULL);
        printf("Parent: One child completed.\n");
    }

    printf("All child processes completed.\n");

    return 0;
}
```

Output:

```
nishanth@LAPTOP-6Q3ONO7H:~/waitlab$ gcc program1.c -o program1
nishanth@LAPTOP-6Q3ONO7H:~/waitlab$ ./program1
Child 1: PID = 694
Child 2: PID = 695
Child 3: PID = 696
Child 1 Finished
Child 2 Finished
Child 3 Finished
Parent: One child completed.
Parent: One child completed.
Parent: One child completed.
All child processes completed.
```

Program2:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main()
{
    pid_t pid[3];
    int i;

    for(i = 0; i < 3; i++)
    {
        pid[i] = fork();

        if(pid[i] == 0)
        {
            printf("Child %d: PID = %d\n", i + 1, getpid());
            sleep(2);
            printf("Child %d Finished\n", i + 1);
            exit(0);
        }
    }

    for(i = 0; i < 3; i++)
    {
        waitpid(pid[i], NULL, 0);
        printf("Parent: Child with PID %d completed.\n", pid[i]);
    }

    printf("All child processes completed.\n");

    return 0;
}
```

Output:

```
nishanth@LAPTOP-6Q3ONO7H:~/waitlab$ gcc program2.c -o program2
nishanth@LAPTOP-6Q3ONO7H:~/waitlab$ ./program2
Child 1: PID = 731
Child 2: PID = 732
Child 3: PID = 733
Child 1 Finished
Child 2 Finished
Child 3 Finished
Parent: Child with PID 731 completed.
Parent: Child with PID 732 completed.
Parent: Child with PID 733 completed.
All child processes completed.
```

Creating a Zombie Process : code

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());
        printf("Child exiting...\n");
        exit(0);
    }
    else if (pid > 0)
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());

        printf("Parent sleeping for 15 seconds...\n");
        sleep(15);

        printf("Parent exiting.\n");
    }
    else
    {
        printf("Fork failed!\n");
    }

    return 0;
}
```

Output:

```
nishanth@LAPTOP-6Q3ONO7H:~/waitlab$ gcc zombie.c -o zombie
/zombienishanth@LAPTOP-6Q3ONO7H:~/waitlab$ ./zombie
Parent Process
PID = 769
Parent sleeping for 15 seconds...
Child Process
PID = 770
Child exiting...
Parent exiting.
```

Eliminating the zombie process:

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t pid = fork();

    if (pid == 0)
    {
        printf("Child Process\n");
        printf("PID = %d\n", getpid());

        printf("Child exiting...\n");
        exit(0);
    }
    else if (pid > 0)
    {
        printf("Parent Process\n");
        printf("PID = %d\n", getpid());

        wait(NULL);

        printf("Child collected successfully.\n");
        printf("No Zombie Process.\n");
    }
    else
    {
        printf("Fork failed!\n");
    }

    return 0;
}
```

Output:

```
nishanth@LAPTOP-6Q3ONO7H:~/waitlab$ gcc no_zombie.c -o no_zombie
/no_zombienishanth@LAPTOP-6Q3ONO7H:~/waitlab$ ./no_zombie
Parent Process
PID = 804
Child Process
PID = 805
Child exiting...
Child collected successfully.
No Zombie Process.
```